CASE STUDY-1: PROGRESSIVE ELECTRICITY BILLING:-

1.PROBLEM STATEMENT:-

Design a Python program to calculate household electricity bills using multiple consumption slabs, fixed charges, surcharges, and rebates. The program compares bills across several months and identifies unusual increases. Python Activity1 case studies.pdf

2.OBJECTIVES:-

1.Read electricity units consumed for different months.

2.Calculate bill using slab rates.

3.Add fixed charges.

4.Apply surcharge and rebate.

5.Compare monthly bills.

6.Identify unusual bill increase.

3.ASSUMPTIONS:-

1.Slab 1 (0–100 Units) → ₹2/unit

2.Slab 2 (101–200 Units) → ₹3/unit

3.Above 200 Units → ₹5/unit

4.Fixed Charge = ₹100

5.Surcharge = 10% (if bill > ₹1000)

6.Rebate = 5% (if units < 100)

4.ALGORITHM:-

1.Start

2.Store monthly unit consumption.

3.Calculate electricity charge according to slab.

4.Add fixed charge.

5.Apply surcharge if required.

6.Apply rebate if eligible.

7.Store final bill.

8.Compare all monthly bills.

9.Display highest bill and unusual increase.

10.Stop

5.PSEUDOCODE:-

START

Store monthly units

FOR each month

    Calculate slab amount

    Add fixed charge

    IF bill >1000

        Add surcharge

    IF units <100

        Apply rebate

Display monthly bills

Compare bills

Display highest bill

STOP

6.FLOWCHART:-

START

↓

Input Monthly Units

↓

Calculate Slab Charges

↓

Add Fixed Charge

↓

Bill >1000?

├──Yes → Add Surcharge

└──No

↓

Units <100?

├──Yes → Apply Rebate

└──No

↓

Store Bill

↓

Compare Bills

↓

Display Results

↓

STOP

7.PYTHON SOURCE CODE:-

```
# Progressive Electricity Billing


months = [
    ("January", 95),
    ("February", 150),
    ("March", 220),
    ("April", 180)
]


bills = []


for month, units in months:


    if units <= 100:
        amount = units * 2


    elif units <= 200:
        amount = (100 * 2) + ((units - 100) * 3)


    else:
```

```
        amount = (100 * 2) + (100 * 3) + ((units - 200) * 5)

    bill = amount + 100

    if bill > 1000:
        bill = bill + bill * 0.10

    if units < 100:
        bill = bill - bill * 0.05

    bills.append((month, units, round(bill,2)))

print("MONTHLY ELECTRICITY BILL")
print("------------------------")

highest = bills[0]

for item in bills:
    print(item)

    if item[2] > highest[2]:
        highest = item

print("------------------------")
print("Highest Bill")
print(highest)
```

8.TEST CASES:-

Test Case

Units

Expected Result

1

95

Rebate Applied

2

150

Normal Bill

3

220

Surcharge Applied

4

180

Normal Bill

9.OUTPUT:-

```
MONTHLY ELECTRICITY BILL
-------------------------
('January', 95, 275.5)
('February', 150, 450)
('March', 220, 700)
('April', 180, 540)
-------------------------
Highest Bill
('March', 220, 700)
```

10. Conclusion

The Progressive Electricity Billing system calculates electricity charges using slab rates, fixed charges, surcharge, and rebate. It compares monthly bills and identifies the highest electricity bill efficiently.

CASE STUDY-2: EMPLOYEE PAYROLL AND DEDUCTION ANALYSIS:-

1.PROBLEM STATEMENT:-

Develop a Python program to calculate an employee's gross salary using basic pay, allowances, overtime, and incentives. Apply deductions and generate a formatted salary statement. Python Activity1 case studies.pdf

2.OBJECTIVES:-

1.Accept employee salary details.

2.Calculate Gross Salary.

3.Apply deductions like PF and Tax.

4.Calculate Net Salary.

5.Display salary statement.

3.ASSUMPTIONS:-

`1.Basic Salary = User Input

2.HRA = 20% of Basic Salary

3.DA = 10% of Basic Salary

4.Overtime = ₹200/hour

5.Incentive = User Input

6.PF Deduction = 12%

7.Tax = 5% (if Gross Salary > ₹30,000)

4.ALGORITHM:-

1.Start

2.Input Basic Salary

3.Input Overtime Hours

4.Input Incentive

5.Calculate HRA

6.Calculate DA

7.Calculate Overtime Pay

8.Calculate Gross Salary

9.Calculate PF

10.If Gross Salary > 30000, calculate Tax

11.Calculate Net Salary

12.Display Salary Statement

13.Stop

5.PSEUDOCODE:-

START

Input Basic Salary

Input Overtime Hours

Input Incentive

HRA = Basic × 20%

DA = Basic × 10%

Overtime = Hours × 200

Gross = Basic + HRA + DA + Overtime + Incentive

PF = Gross × 12%

IF Gross > 30000

Tax = Gross × 5%

ELSE

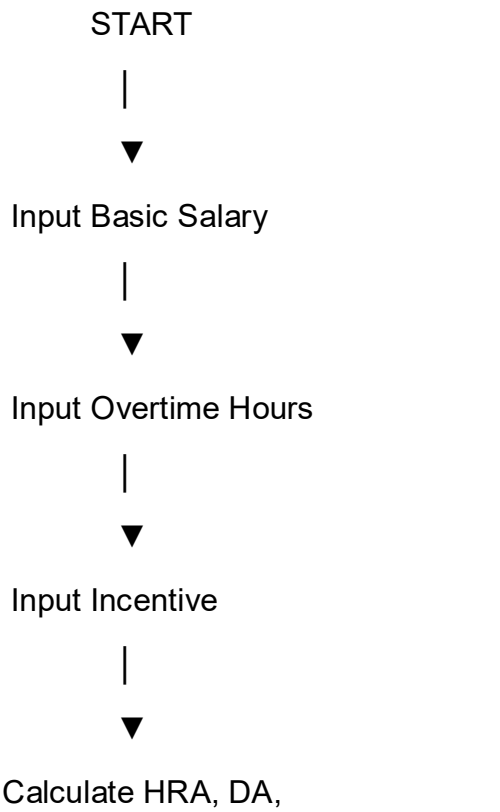
Tax = 0

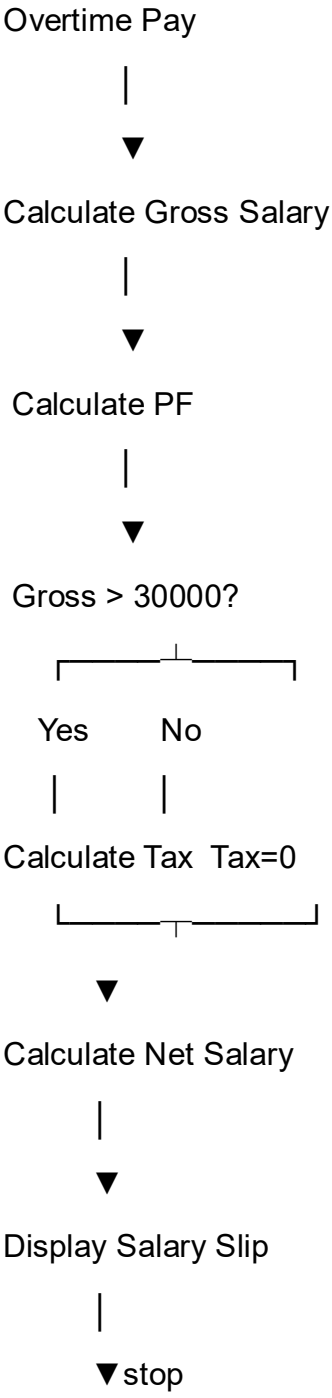
Net Salary = Gross − PF − Tax

Display Salary Details

STOP

6.FLOWCHART:-





Employee Payroll and Deduction Analysis

7.python source code:-

```
basic = float(input("Enter Basic Salary: "))
```

```
hours = int(input("Enter Overtime Hours: "))
incentive = float(input("Enter Incentive: "))

hra = basic * 0.20
da = basic * 0.10
overtime = hours * 200

gross = basic + hra + da + overtime + incentive

pf = gross * 0.12

if gross > 30000:
    tax = gross * 0.05
else:
    tax = 0

net = gross - pf - tax

print("\n------ EMPLOYEE SALARY STATEMENT ------")
print("Basic Salary :", basic)
print("HRA          :", hra)
print("DA           :", da)
print("Overtime Pay :", overtime)
print("Incentive    :", incentive)
print("Gross Salary :", gross)
print("PF Deduction :", pf)
print("Tax          :", tax)
```

```
print("Net Salary   :", net)
```

8.TEST CASES:-

Test Case	Basic Salary	Overtime	Incentive	Expected Result
1	20000	5	1000	Tax Not Applied
2	35000	4	2000	Tax Applied
3	28000	0	500	Normal Salary
4	40000	10	5000	

High salary.

9.output:-

```
Enter Basic Salary: 30000
Enter Overtime Hours: 5
Enter Incentive: 2000

------ EMPLOYEE SALARY STATEMENT ------
Basic Salary : 30000.0
HRA          : 6000.0
DA           : 3000.0
Overtime Pay : 1000
Incentive    : 2000.0
Gross Salary : 42000.0
PF Deduction : 5040.0
Tax          : 2100.0
Net Salary   : 34860.0
```

10. Conclusion

This Employee Payroll and Deduction Analysis program calculates the gross salary by including allowances, overtime pay, and incentives. It applies PF and tax deductions according to the specified conditions and generates a clear salary statement for the employee.

CASE STUDY-3: RETAIL BILLING WITH MULTIPLE DISCOUNT RULES:-

1.PROBLEM STATEMENT:-

Develop a Python program to process multiple products, quantities, category-wise taxes, threshold discounts, and maximum concession limits to generate an itemized retail invoice. Python Activity1 case studies.pdf

2.OBJECTIVES:-

1.Accept multiple product details.

2.Calculate item-wise total amount.

3.Apply category-wise GST.

4.Apply discount based on bill amount.

5.Generate itemized invoice.

6.Display final payable amount.

3.ASSUMPTIONS:-

1.GST Rates:

Grocery = 5%

Electronics = 18%

Clothing = 12%

2.Discount:

Bill ≥ ₹5000 → 10%

Bill ≥ ₹3000 → 5%

Otherwise → No Discount

3.Maximum Discount = ₹1000

4.ALGORITHM:-

1.Start

2.Read number of products.

3.Input product name, category, price and quantity.

4.Calculate item total.

5.Apply GST according to category.

6.Add all item totals.

7.Apply eligible discount.

8.Limit discount to ₹1000 if required.

9.Display invoice.

10.Stop.

5.PSEUDOCODE:-

START

Input Number of Products

FOR each Product

    Input Name

    Input Category

    Input Price

    Input Quantity

    Total = Price × Quantity

    Apply GST

    Add to Grand Total

END FOR

IF Grand Total >=5000

    Discount =10%

ELSE IF Grand Total >=3000

    Discount =5%

ELSE

Discount=0

IF Discount >1000

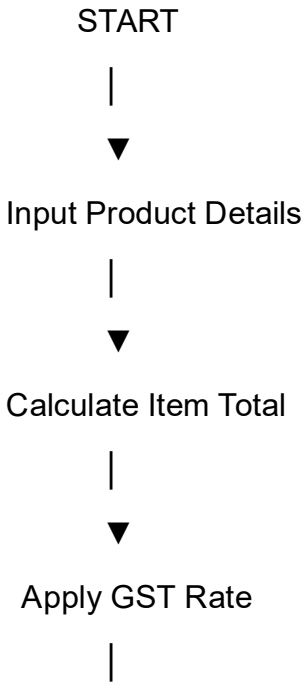
Discount=1000

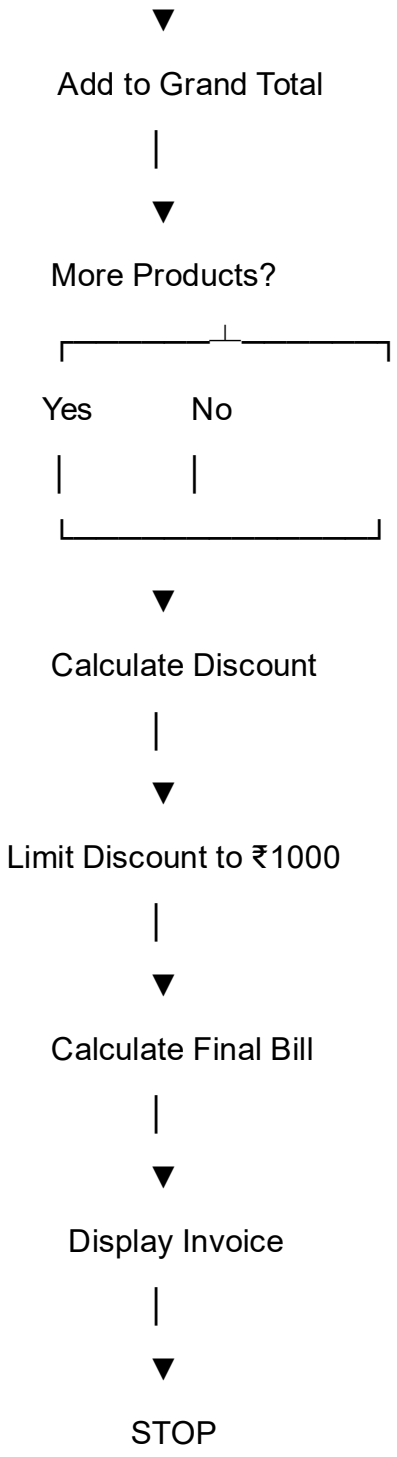
Final Bill = Grand Total − Discount

Display Invoice

STOP

6.FLOWCHART:-





7.PYTHON SOURCE CODE:-

```
# Retail Billing with Multiple Discount Rules
```

```
n = int(input("Enter Number of Products: "))

grand_total = 0

for i in range(n):

    print("\nProduct", i + 1)

    name = input("Product Name: ")
    category = input("Category (Grocery/Electronics/Clothing): ")
    price = float(input("Price: "))
    qty = int(input("Quantity: "))

    total = price * qty

    if category.lower() == "grocery":
        gst = total * 0.05

    elif category.lower() == "electronics":
        gst = total * 0.18

    elif category.lower() == "clothing":
        gst = total * 0.12

    else:
        gst = 0
```

```
    amount = total + gst

    grand_total += amount

    print("Item Amount =", amount)

if grand_total >= 5000:
    discount = grand_total * 0.10

elif grand_total >= 3000:
    discount = grand_total * 0.05

else:
    discount = 0

if discount > 1000:
    discount = 1000

final_bill = grand_total - discount

print("\n-------- RETAIL BILL --------")
print("Grand Total :", round(grand_total,2))
print("Discount    :", round(discount,2))
print("Final Bill  :", round(final_bill,2))
```

8. Test Cases

Test Case

Input

Expected Result

1

Rice ₹60 × 20

5% GST Applied

2

TV ₹25000 × 1

18% GST + 10% Discount

3

Shirt ₹800 × 4

12% GST + 5% Discount

4

Books ₹500 × 2

No GST, No Discount

9.OUTPUT:-

```
>>
=============== RESTART: C:/Users/bezaw/OneDrive/Desktop/padma.py ==============
Enter Number of Products: 2

Product 1
Product Name: Rice
Category (Grocery/Electronics/Clothing): Grocery
Price: 60
Quantity: 20
Item Amount = 1260.0

Product 2
Product Name: TV
Category (Grocery/Electronics/Clothing): Electronics
Price: 25000
Quantity: \1
Item Amount = 29500.0

-------- RETAIL BILL --------
Grand Total : 30760.0
Discount    : 1000
Final Bill  : 29760.0
```

10.CONCLUSION:-

The Retail Billing with Multiple Discount Rules program successfully calculates item-wise billing, applies category-based GST, provides threshold discounts with a maximum concession limit, and generates an accurate retail invoice. This program demonstrates the use of variables, conditional statements, loops, operators, and formatted output

CASE STUDY-4: STUDENT EMAIL AND IDENTIFIER VALIDATION:-

1.PROBLEM STATEMENT:-

Develop a Python program to validate student email-like strings and registration identifiers based on the given formatting rules. The program should detect malformed separators, invalid character groups, and missing components. Python Activity1 case studies.pdf

2. OBJECTIVES:-

1.Accept a student email ID.

2.Accept a registration number.

3.Validate the email format.

4.Validate the registration ID format.

5.Display whether each input is valid or invalid.

6.Report the reason for invalid input.

3.ASSUMPTIONS:-

1.Email must contain exactly one @ symbol.

2.Email must end with .com.

3.Username should not be empty.

4.Registration ID format: 22CSE1234

First 2 digits → Admission Year

Next 3 letters → Branch

Last 4 digits → Roll Number

4.ALGORITHM:-

1.Start.

2.Input email and registration ID.

3.Check whether email contains exactly one @.

4.Check whether email ends with .com.

5.Validate registration ID length.

6.Check first 2 characters are digits.

7.Check next 3 characters are alphabets.

8.Check last 4 characters are digits.

9.Display validation results.

10.Stop.

5.PSEUDOCODE:-

```
START

Input Email
Input Registration ID

IF Email contains one '@'
  AND Email ends with '.com'
     Email is Valid
ELSE
     Email is Invalid
```

IF Registration length = 9

First 2 are digits

Next 3 are letters

Last 4 are digits

Registration ID is Valid

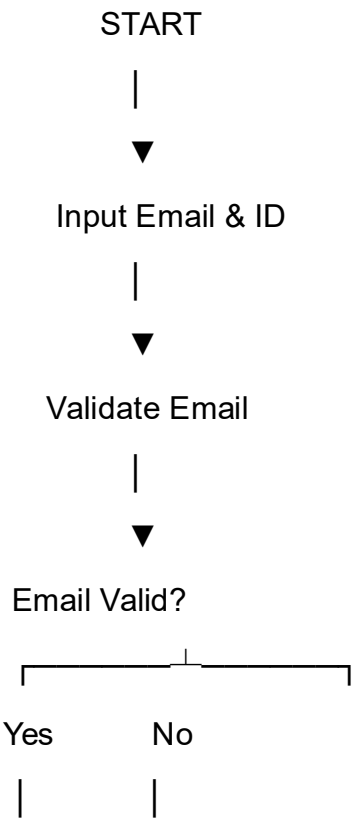
ELSE

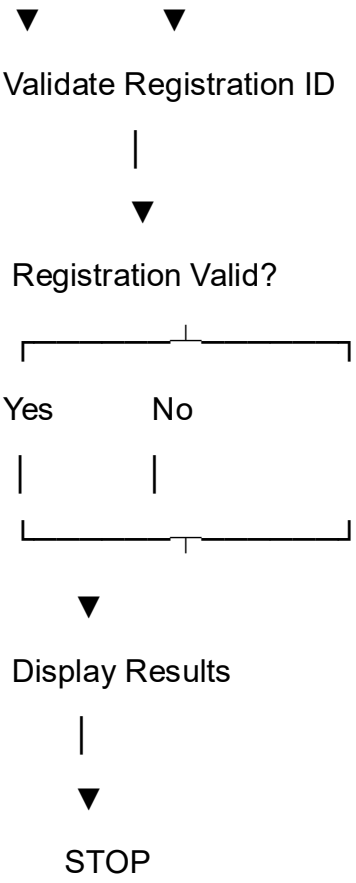
Registration ID is Invalid

Display Results

STOP

6.FLOWCHART:-





PYTHON SOURCE CODE:-

```
# Student Email and Identifier Validation

email = input("Enter Student Email: ")
reg = input("Enter Registration ID: ")

# Email Validation
if email.count("@") == 1 and email.endswith(".com"):
    print("Email : Valid")
else:
    print("Email : Invalid")
```

```
# Registration Validation
if len(reg) == 9:
    year = reg[:2]
    branch = reg[2:5]
    roll = reg[5:]

    if year.isdigit() and branch.isalpha() and roll.isdigit():
        print("Registration ID : Valid")
    else:
        print("Registration ID : Invalid")
else:
    print("Registration ID : Invalid")
```

8.TEST CASES:-

Test Case	Email	Registration ID	Expected Result
1	student@gmail.com	22CSE1234	Both Valid
2	studentgmail.com	22CSE1234	Invalid Email
3			

abc@gmail.com

22CS1234

Invalid Registration

4

abc@yahoo.com

23ECE4567

Both Valid

9.OUTPUT:-

```
--------------- RESTART: C:/Users/bezaw/OneDrive/Desktop/padma.py ---------------
Enter Student Email: bezawadaprabha296@g mail.com
Enter Registration ID: 26001573
Email : Valid
Registration ID : Invalid

=============== RESTART: C:/Users/bezaw/OneDrive/Desktop/padma.py ===============
Enter Student Email: student@g mail.com
Enter Registration ID: 22cs1234
Email : Valid
Registration ID : Invalid
```

10.OUTPUT:-

The Student Email and Identifier Validation program validates student email addresses and registration IDs using basic string operations, conditional statements, and built-in string functions. It detects invalid formats and reports whether the entered details are valid, making it useful for student record verification.

CASE STUDY-5: NESTED LIST CLASSROOM OCCUPANCY ANALYSIS:-

1.PROBLEM STATEMENT:-

Develop a Python program to represent multiple classrooms and seating positions using nested lists. The program should allocate students, preserve reserved seats, detect duplicate assignments, and calculate vacant capacity. Python Activity1 case studies.pdf

2.OBJECTIVES:-

1.Represent classroom seating using nested lists.

2.Allocate students to seats.

3.Preserve reserved seats.

4.Detect duplicate student allocations.

5.Calculate vacant seats.

6.Display the classroom seating arrangement.

3.ASSUMPTIONS:-

1.Classroom has 3 rows × 4 columns.

2.Reserved seats are represented by "R".

3.Empty seats are represented by "-".

4.Student roll numbers are unique.

5.Duplicate roll numbers are not allowed.

4.ALGORITHM:-

1.Start.

2.Create a nested list representing classroom seats.

3.Display the seating arrangement.

5.Count reserved seats.

6.Count vacant seats.

7.Check for duplicate student roll numbers.

8.Display duplicate status.

9.Display total vacant capacity.

10.Stop.

5.PSEUDOCODE:-

START

Create Nested List

Display Seating Arrangement

Vacant = 0

Reserved = 0

FOR each row

    FOR each seat

        IF seat == "-"

            Vacant++

        ELSE IF seat == "R"

            Reserved++

        ELSE

            Check Duplicate

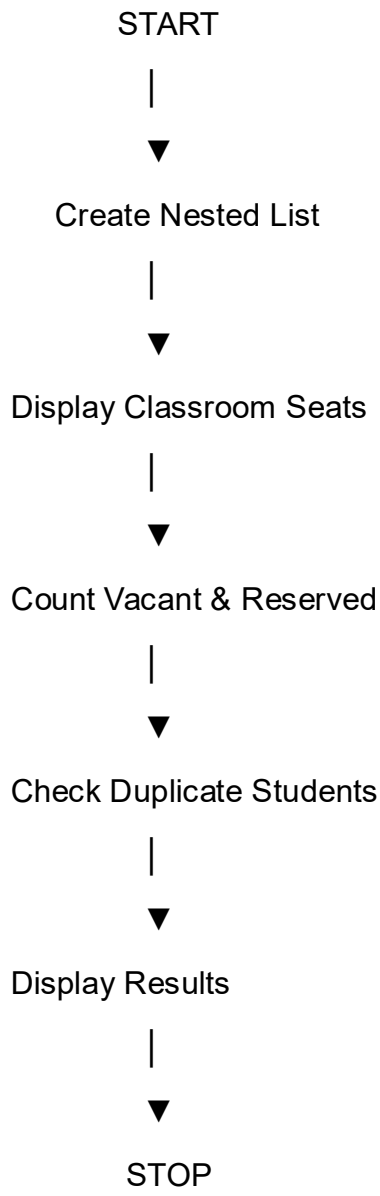
Display Reserved Seats

Display Vacant Seats

Display Duplicate Status

STOP

6.FLOWCHART:-



7.PYTHON SOURCE CODE:-

```
# Nested List Classroom Occupancy Analysis

classroom = [
    ["22CSE101", "22CSE102", "-", "R"],
```

```
    ["22CSE103", "-", "22CSE104", "-"],
    ["22CSE105", "22CSE106", "-", "22CSE106"]
]

vacant = 0
reserved = 0
students = []
duplicate = False

print("CLASSROOM SEATING")

for row in classroom:
    print(row)

for row in classroom:
    for seat in row:

        if seat == "-":
            vacant += 1

        elif seat == "R":
            reserved += 1

        else:
            if seat in students:
                duplicate = True
            else:
```

```
            students.append(seat)

print("\nReserved Seats :", reserved)
print("Vacant Seats :", vacant)

if duplicate:
    print("Duplicate Student Allocation Found")
else:
    print("No Duplicate Allocation")
```

8.TEST CASES:-

Test Case	Input	Expected Result
1	Normal seating	No duplicate
2	Reserved seats present	Reserved count displayed
3	Vacant seats present	Vacant count displayed
4	Duplicate roll number	Duplicate detected

9.OUTPUT:-

```
========================================= RESTART: C:/Users/bezaw/OneDrive/Desktop/padma.py ========================
CLASSROOM SEATING
['22CSE101', '22CSE102', '-', 'R']
['22CSE103', '-', '22CSE104', '-']
['22CSE105', '22CSE106', '-', '22CSE106']

Reserved Seats : 1
Vacant Seats : 4
Duplicate Student Allocation Found
```

10.CONCLUSION:-

The Nested List Classroom Occupancy Analysis program uses nested lists, loops, conditional statements, and lists to manage classroom seating. It successfully counts vacant and reserved seats, detects duplicate student allocations, and displays the seating arrangement, making it suitable for classroom management.

CASE STUDY-6:
FOOD WASTE AND MONITORING SYSTEM:-

1.PROBLEM STATEMENT:-

Develop a Python program to validate student email-like strings and registration identifiers based on the given formatting rules. The program should detect malformed separators, invalid character groups, and missing components. Python Activity1 case studies.pdf

2.OBJECTIVES:-

1.Accept a student email ID.

2.Accept a registration number.

3.Validate the email format.

4.Validate the registration ID format.

5.Display whether each input is valid or invalid.

6.Report the reason for invalid input.

3.ASSUMPTIONS:-

1.Email must contain exactly one @ symbol.

2.Email must end with .com.

3.Username should not be empty.

4.Registration ID format: 22CSE1234

5.First 2 digits → Admission Year

6.Next 3 letters → Branch

7.Last 4 digits → Roll Number

4.ALGORITHM:-

1.Start.

2.Input email and registration ID.

3.Check whether email contains exactly one @.

4.Check whether email ends with .com.

5.Validate registration ID length.

6.Check first 2 characters are digits.

7.Check next 3 characters are alphabets.

8.Check last 4 characters are digits.

9.Display validation results.

10.Stop.

5.PSEUDOCODE:-

START

Input Email

Input Registration ID

IF Email contains one '@'

AND Email ends with '.com'

Email is Valid

ELSE

Email is Invalid

IF Registration length = 9

First 2 are digits

Next 3 are letters

Last 4 are digits

Registration ID is Valid

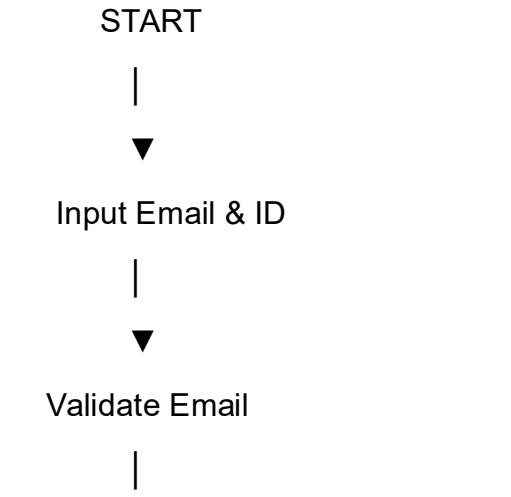
ELSE

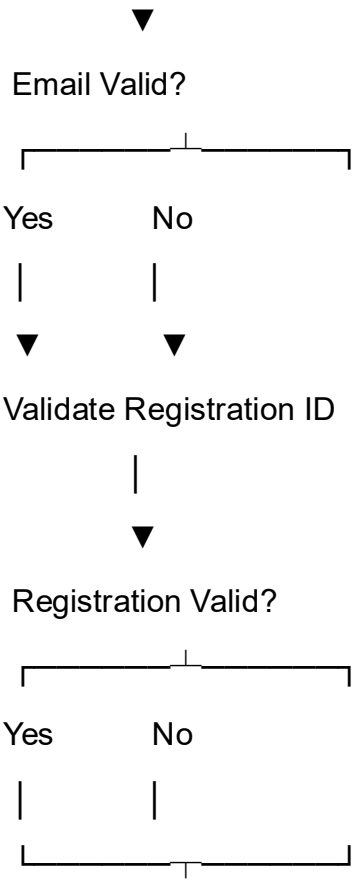
Registration ID is Invalid

Display Results

STOP

6.FLOW CHART:-





7.PYTHON SOURCE CODE:-

```
# Student Email and Identifier Validation


email = input("Enter Student Email: ")

reg = input("Enter Registration ID: ")


# Email Validation
```

```
if email.count("@") == 1 and email.endswith(".com"):
    print("Email : Valid")
else:
    print("Email : Invalid")

# Registration Validation
if len(reg) == 9:
    year = reg[:2]
    branch = reg[2:5]
    roll = reg[5:]

    if year.isdigit() and branch.isalpha() and roll.isdigit():
        print("Registration ID : Valid")
    else:
        print("Registration ID : Invalid")
else:
    print("Registration ID : Invalid")
```

8.TEST CASES:-

Test Case	Email	Registration ID	Expected Result
1	student@gmail.com	22CSE1234	Both Valid

2

studentgmail.com

22CSE1234

Invalid Email

3

abc@gmail.com

22CS1234

Invalid Registration

4

abc@yahoo.com

23ECE4567

Both Valid

9.OUTPUT:-

```
========================================= RESTART: C:/Users/bezaw/OneDrive/Desktop/padma.py ==============================
FOOD WASTE REPORT
-------------------------------
Day 1
Prepared : 500
Served   : 470
Waste    : 30
Waste %  : 6.0
Status : Within Limit
-------------------------------
Day 2
Prepared : 450
Served   : 420
Waste    : 30
Waste %  : 6.67
Status : Within Limit
-------------------------------
Day 3
Prepared : 480
Served   : 460
Waste    : 20
Waste %  : 4.17
Status : Within Limit

--------------------------------
Day 4
Prepared : 520
Served   : 500
Waste    : 20
Waste %  : 3.85
Status : Within Limit
--------------------------------
Day 5
Prepared : 500
Served   : 430
Waste    : 70
Waste %  : 14.0
Status : Excessive Waste
```

10.CONCLUSION:-

The Student Email and Identifier Validation program validates student email addresses and registration IDs using basic string operations, conditional statements, and built-in string functions. It detects invalid formats and reports whether the entered details are valid, making it useful for student record verification.

CASE STUDY-7: TUPLE -BASED COURSE REGISTRATION VALIDATION:-

1.PROBLEM STATEMENT:-

Develop a Python program to represent course registrations using tuples, identify duplicate registrations and invalid course combinations, and determine students whose course selections violate the given rules. Python Activity1 case studies.pdf

2.OBJECTIVES:-

1.Store student course registrations using tuples.

2.Detect duplicate registrations.

3.Validate course combinations.

4.Identify students with invalid registrations.

5.Display registration status.

3.ASSUMPTIONS:-

1.Each record is stored as a tuple: (Student_ID, Course1, Course2)

2.A student cannot register for the same course twice.

3.Invalid combination:

Python and Java Lab cannot be selected together.

Duplicate Student IDs are not allowed.

4.ALGORITHM:-

1.Start.

2.Create tuples containing student registration details.

3.Check for duplicate Student IDs.

4.Check whether the same course is selected twice.

5.Check for invalid course combinations.

6.Display validation results.

7.Stop.

5.PSEUDOCODE:-

START

Create Registration Tuples

Create Empty Student List

FOR each Registration

IF Student ID already exists

Display Duplicate Registration

ELSE

Store Student ID

IF Course1 = Course2

Display Invalid Course Selection

IF Course1 = Python AND Course2 = Java Lab

OR

Course1 = Java Lab AND Course2 = Python

Display Invalid Combination

END FOR

STOP

6.FLOWCHART:-

START

|

▼

Create Registration Tuples

|

▼

Check Duplicate Student ID

|

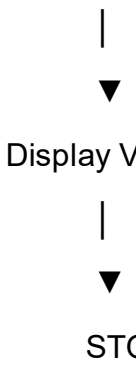
▼

Check Same Course Selected

|

▼

Check Invalid Course Combination



7.PYTHON SOURCE CODE:-

```
# Tuple-Based Course Registration Validation

registrations = [
    ("22CSE101", "Python", "Data Science"),
    ("22CSE102", "Python", "Java Lab"),
    ("22CSE103", "C Programming", "Python"),
    ("22CSE101", "Java", "DBMS"),
    ("22CSE104", "Python", "Python")
]

students = []

print("COURSE REGISTRATION REPORT")
print("---------------------------")

for record in registrations:

    sid, course1, course2 = record
```

```
    if sid in students:
        print(sid, ": Duplicate Registration")
    else:
        students.append(sid)

    if course1 == course2:
        print(sid, ": Invalid - Same Course Selected Twice")

    elif (course1 == "Python" and course2 == "Java Lab") or \
         (course1 == "Java Lab" and course2 == "Python"):
        print(sid, ": Invalid Course Combination")

    else:
        print(sid, ": Registration Valid")
```

8.TEST CASES:-

Test Case

Student ID

Courses

Expected Result

1

22CSE101

Python, Data Science

Valid

2

22CSE102

Python, Java Lab

Invalid Combination

3

22CSE103

C Programming, Python

Valid

4

22CSE101

Java, DBMS

Duplicate Registration

5

22CSE104

Python, Python

Invalid (Same Course)

9.OUTPUT:-

```
========================================= RESTART: C:/Users/bezaw/OneDrive/Desktop/padma.py ==========================
COURSE REGISTRATION REPORT
---------------------------
22CSE101 : Registration Valid
22CSE102 : Invalid Course Combination
22CSE103 : Registration Valid
22CSE101 : Duplicate Registration
22CSE101 : Registration Valid
22CSE104 : Invalid - Same Course Selected Twice
```

10.CONCLUSION:-

The Tuple-Based Course Registration Validation program uses tuples, tuple unpacking, lists, loops, and conditional statements to validate student course registrations. It successfully detects duplicate registrations, invalid course selections, and prohibited course combinations, making it suitable for university course registration management.

CASE STUDY-8: SMART HOSTEL RESOURCE MONITORING:-

1.PROBLEM STATEMENT:-

Develop a Python program to analyze room-wise electricity and water consumption in a hostel, calculate charges, identify rooms exceeding the specified limits, and compare resource utilization across different floors. Python Activity1 case studies.pdf

2.OBJECTIVES:-

1.Store room-wise electricity and water consumption.

2.Calculate electricity and water charges.

3.Find the total resource usage for each room.

4.Identify rooms exceeding the usage limit.

5.Compare total resource consumption floor-wise.

6.Display a complete hostel resource report.

3.ASSUMPTIONS:-

1.Electricity Charge = ₹8 per unit

2.Water Charge = ₹15 per KL

3.Electricity Limit = 200 units

4.Water Limit = 25 KL

Hostel has 2 floors, each containing 3 rooms.

4.ALGORITHM:-

1.Start.

2.Store room-wise electricity and water usage using nested lists.

3.Calculate electricity and water charges for each room.

4.Calculate total bill.

5.Check whether electricity or water usage exceeds the limit.

6.Calculate total usage for each floor.

7.Display the report.

8.Stop.

5.PSEUDOCODE:-

```
START

Store Electricity Data
Store Water Data

FOR each Floor

    FOR each Room

        Electricity Charge = Units × 8
        Water Charge = KL × 15

        Total = Electricity Charge + Water Charge

        IF Units > 200 OR Water > 25

            Display "Limit Exceeded"

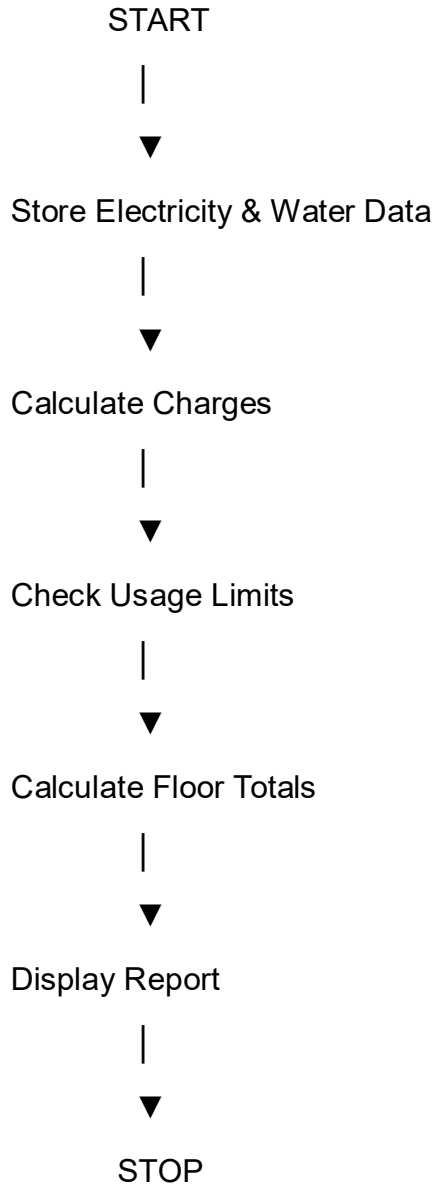
        ELSE

            Display "Within Limit"

Display Floor-wise Usage
```

STOP

6.FLOWCHART:-



7.PYTHON SOURCE CODE:-

```
# Smart Hostel Resource Monitoring


electricity = [
    [180, 210, 195],
```

```
    [160, 220, 175]
]

water = [
    [20, 28, 22],
    [18, 30, 24]
]

print("HOSTEL RESOURCE REPORT")
print("-------------------------------")

for floor in range(len(electricity)):

    floor_total = 0

    print("\nFloor", floor + 1)

    for room in range(len(electricity[floor])):

        units = electricity[floor][room]
        water_used = water[floor][room]

        e_charge = units * 8
        w_charge = water_used * 15

        total = e_charge + w_charge
```

```
        floor_total += total

        print("Room", room + 1)
        print("Electricity Units :", units)
        print("Water Used :", water_used, "KL")
        print("Total Bill :", total)

        if units > 200 or water_used > 25:
            print("Status : Limit Exceeded")
        else:
            print("Status : Within Limit")

        print("----------------------")

    print("Floor Total Bill :", floor_total)
```

8.TEST CASES:-

Test Case	Electricity Units	Water (KL)	Expected Result
1	180	20	Within Limit
2	210		

28

Limit Exceeded

3

195

22

Within Limit

4

220

30

Limit Exceeded

5

175

24

Within Limit

9.OUTPUT:-

```
HOSTEL RESOURCE REPORT
--------------------------------

Floor 1
Room 1
Electricity Units : 180
Water Used : 20 KL
Total Bill : 1740
Status : Within Limit
----------------------
Room 2
Electricity Units : 210
Water Used : 28 KL
Total Bill : 2100
Status : Limit Exceeded
----------------------
Room 3
Electricity Units : 195
Water Used : 22 KL
Total Bill : 1890
Status : Within Limit
----------------------
Floor Total Bill : 5730
```

```
Floor 2
Room 1
Electricity Units : 160
Water Used : 18 KL
Total Bill : 1550
Status : Within Limit
-----------------------
Room 2
Electricity Units : 220
Water Used : 30 KL
Total Bill : 2210
Status : Limit Exceeded
-----------------------
Room 3
Electricity Units : 175
Water Used : 24 KL
Total Bill : 1760
Status : Within Limit
-----------------------
Floor Total Bill : 5520
```

10.CONCLUSION:-

The Smart Hostel Resource Monitoring program uses nested lists, loops, conditional statements, and arithmetic operators to monitor room-wise electricity and water consumption. It calculates utility charges, identifies rooms exceeding specified limits, and compares floor-wise resource utilization, making it suitable for hostel resource management.

CASE STUDY-9: CAMPUS EVENT SEATING AND ENTRY VALIDATION :-

1.PROBLEM STATEMENT:-

Develop a Python program to generate seating assignments using nested lists, validate tickets against registration identifiers, detect duplicate entries, and report the available seating capacity for a campus event. Python Activity1 case studies.pdf

2.OBJECTIVES:-

1.Create seating arrangements using nested lists.

2.Validate ticket IDs with registered student IDs.

3.Detect duplicate entries.

4.Calculate vacant seats.

5.Display the final seating report.

3.ASSUMPTIONS:-

1.Event Hall has 3 rows × 4 seats.

2.Empty seats are represented by "-".

3.Registered student IDs are unique.

4.Duplicate ticket IDs are not allowed.

Total seating capacity = 12 seats.

4.ALGORITHM:-

1.Start.

2.Create a nested list for seating arrangement.

3.Store registered student IDs.

4.Check each seat:

If seat is empty, count as vacant.

If student ID is already entered, report duplicate.

If student ID is not registered, report invalid entry.

5.Display seating arrangement.

6.Display duplicate entries and vacant seats.

7.Stop.

5.PSEUDOCODE:-

START

Create Registered Student List

Create Seating Matrix

Vacant = 0

FOR each Row

FOR each Seat

IF Seat == "-"

Vacant++

ELSE IF Seat not in Registered List

Display Invalid Entry

ELSE IF Seat already checked

Display Duplicate Entry

Display Seating Report

Display Vacant Seats

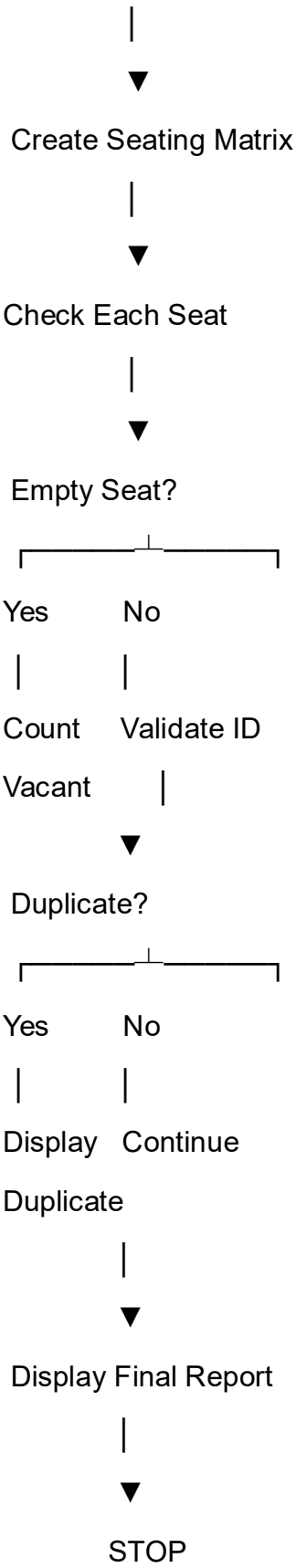
STOP

6.FLOWCHART:-

START



Create Registered List


Create Seating Matrix
Check Each Seat
Empty Seat?
Yes
No
Count Vacant
Validate ID
Duplicate?
Yes
No
Display Duplicate
Continue
Display Final Report
STOP

7.PYTHON SOURCE CODE:-

```
# Campus Event Seating and Entry Validation

registered = [
    "22CSE101", "22CSE102", "22CSE103",
    "22CSE104", "22CSE105", "22CSE106"
]

seating = [
    ["22CSE101", "22CSE102", "-", "22CSE103"],
    ["22CSE104", "22CSE102", "22CSE105", "-"],
    ["22CSE106", "-", "-", "22CSE110"]
]

checked = []
vacant = 0

print("EVENT SEATING REPORT")
print("-------------------------")

for row in seating:
    print(row)

print("\nValidation Report")

for row in seating:
    for seat in row:
```

```
        if seat == "-":
            vacant += 1

        else:

            if seat in checked:
                print(seat, ": Duplicate Entry")

            else:
                checked.append(seat)

            if seat not in registered:
                print(seat, ": Invalid Registration")

print("\nVacant Seats :", vacant)
print("Occupied Seats :", 12 - vacant)
```

8.TEST CASES:-

Test Case	Input	Expected Result
1	Registered ID	Valid Entry
2	Duplicate ID	

Duplicate Entry

3

Unregistered ID

Invalid Registration

4

Empty Seat (-)

Vacant Seat Count

9.OUTPUT:-

```
========================================= RESTART: C:/Users/bezaw/OneDrive/Desktop/padma.py ==============================
EVENT SEATING REPORT
------------------------
['22CSE101', '22CSE102', '-', '22CSE103']
['22CSE104', '22CSE102', '22CSE105', '-']
['22CSE106', '-', '-', '22CSE110']

Validation Report
22CSE102 : Duplicate Entry
22CSE110 : Invalid Registration

Vacant Seats : 4
Occupied Seats : 8
```

10.CONCLUSION:-

The Campus Event Seating and Entry Validation program uses nested lists, loops, lists, and conditional statements to validate event seating. It detects duplicate ticket entries, identifies unregistered participants, calculates available seating capacity, and generates a complete seating report. This program demonstrates the effective use of nested lists and validation techniques in Python.

CASE STUDY-10:
SMART PUBLIC TRANSPORT FARE RECONCILIATION:-

1.PROBLEM STATEMENT:-

Develop a Python program to generate seating assignments using nested lists, validate tickets against registration identifiers, detect duplicate entries, and report the available seating capacity for a campus event. Python Activity1 case studies.pdf

2.OBJECTIVES:-

1.Create seating arrangements using nested lists.

2.Validate ticket IDs with registered student IDs.

3.Detect duplicate entries.

4.Calculate vacant seats.

5.Display the final seating report.

3.ASSUMPTIONS:-

1.Event Hall has 3 rows × 4 seats.

2.Empty seats are represented by "-".

3.Registered student IDs are unique.

4.Duplicate ticket IDs are not allowed.

5.Total seating capacity = 12 seats.

4.ALGORITHM:-

1.Start.

2.Create a nested list for seating arrangement.

3.Store registered student IDs.

4.Check each seat:

If seat is empty, count as vacant.

If student ID is already entered, report duplicate.

If student ID is not registered, report invalid entry.

5.Display seating arrangement.

6.Display duplicate entries and vacant seats.

7.Stop.

5.PSEUDOCODE:-

START

```
Create Registered Student List

Create Seating Matrix

Vacant = 0

FOR each Row

    FOR each Seat

        IF Seat == "-"
            Vacant++

        ELSE IF Seat not in Registered List
            Display Invalid Entry

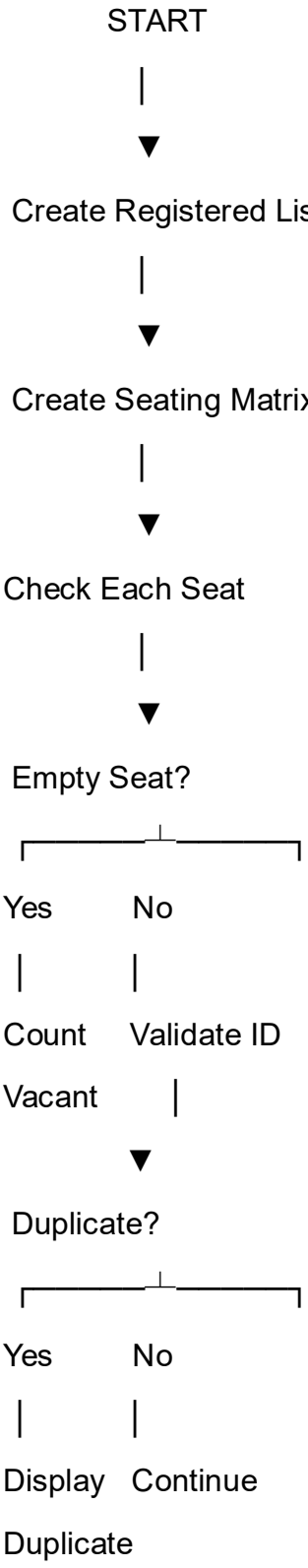
        ELSE IF Seat already checked
            Display Duplicate Entry

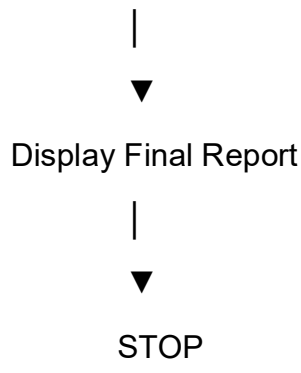
Display Seating Report

Display Vacant Seats

STOP
```

6.FLOWCHART:-


START
Create Registered List
Create Seating Matrix
Check Each Seat
Empty Seat?
Yes
No
Count Vacant
Validate ID
Duplicate?
Yes
No
Display Duplicate
Continue



7.PYTHON SOURCE CODE:-

```
# Campus Event Seating and Entry Validation

registered = [
    "22CSE101", "22CSE102", "22CSE103",
    "22CSE104", "22CSE105", "22CSE106"
]

seating = [
    ["22CSE101", "22CSE102", "-", "22CSE103"],
    ["22CSE104", "22CSE102", "22CSE105", "-"],
    ["22CSE106", "-", "-", "22CSE110"]
]

checked = []
vacant = 0

print("EVENT SEATING REPORT")
print("------------------------")
```

```
for row in seating:
    print(row)

print("\nValidation Report")

for row in seating:
    for seat in row:

        if seat == "-":
            vacant += 1

        else:

            if seat in checked:
                print(seat, ": Duplicate Entry")

            else:
                checked.append(seat)

            if seat not in registered:
                print(seat, ": Invalid Registration")

print("\nVacant Seats :", vacant)
print("Occupied Seats :", 12 - vacant)
```

8.TEST CASES:-

Test Case

Input

Expected Result

1

Registered ID

Valid Entry

2

Duplicate ID

Duplicate Entry

3

Unregistered ID

Invalid Registration

4

Empty Seat (-)

Vacant Seat Count

9. Sample Output

EVENT SEATING REPORT

['22CSE101', '22CSE102', '-', '22CSE103']

['22CSE104', '22CSE102', '22CSE105', '-']

['22CSE106', '-', '-', '22CSE110']

Validation Report

22CSE102 : Duplicate Entry

22CSE110 : Invalid Registration

Vacant Seats : 4

Occupied Seats :

9.OUTPUT:-

```
========== RESTART: C:/Users/bezaw/OneDrive/Desktop/padma.py ==========
EVENT SEATING REPORT
------------------------
['22CSE101', '22CSE102', '-', '22CSE103']
['22CSE104', '22CSE102', '22CSE105', '-']
['22CSE106', '-', '-', '22CSE110']

Validation Report
22CSE102 : Duplicate Entry
22CSE110 : Invalid Registration

Vacant Seats : 4
Occupied Seats : 8
```

10.CONCLUSION:-

The Campus Event Seating and Entry Validation program uses nested lists, loops, lists, and conditional statements to validate event seating. It detects duplicate ticket entries, identifies unregistered participants, calculates available seating capacity, and generates a complete seating report. This program demonstrates the effective use of nested lists and validation techniques in Python.